



MERRIMACK COLLEGE

CSC 6303

Week 2

From Python to Java and C/C++

Systems and Languages Survey - Dr. Paulo Fernandes

Presentation Agenda

Week 2

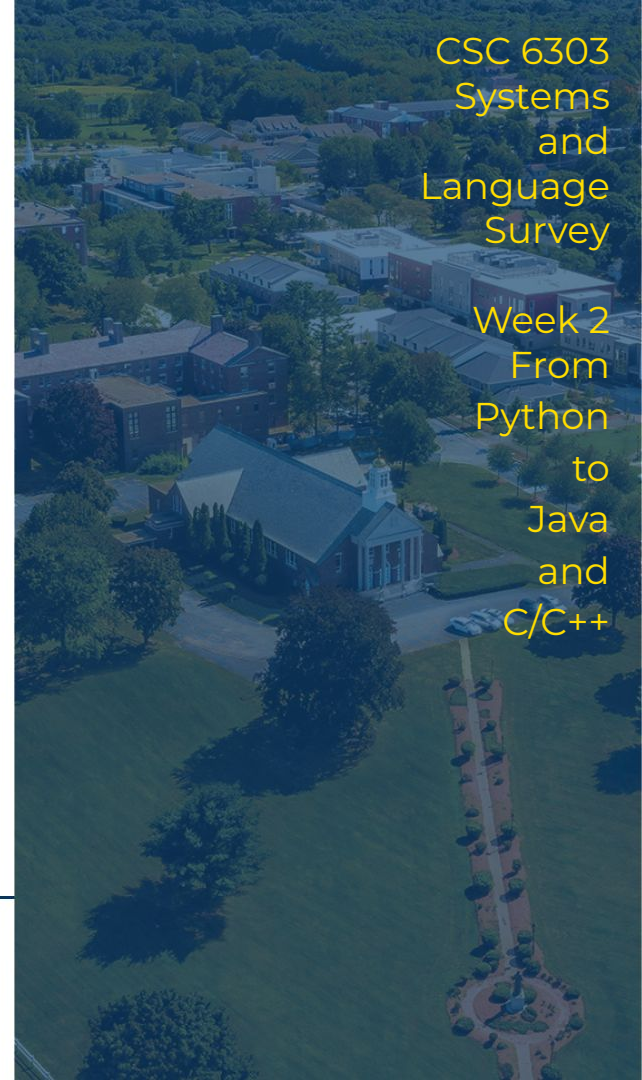
- Using C/C++ and Java
 - a. Online or With and IDE
- Visible Differences
 - a. Program Structure
 - b. Basic Commands
 - c. Classes, Objects, etc.
- Subtle Differences
 - a. Packages, Documentation, and Resources
 - b. Internal Implementation
- This Week's tasks



MERRIMACK COLLEGE

CSC 6303
Systems
and
Language
Survey

Week 2
From
Python
to
Java
and
C/C++



Using C/C++ and Java

You can use it online or have it
installed in your own machine
available on an IDE

Likely your machine already has
both C++ installed for Operating
System sake, as well as Java
installed for running programs
requested by web browsers.

However, for small C++ and Java
programs it is just as easy to use
online servers. Here you will see
some options to use such servers
as well as set running environment
for these languages.



Running code online

Among the several online options you can try these ones:

You can try it on one of the following
C++ interpreters online:

[OnlineGDB](#) - [W3S](#) - [CPPsh](#)

You can try it on one of the following
Java interpreters online:

[OnlineGDB](#) - [JDoodle](#) - [Online Java](#)

A curious thing is that such online servers always behave as interpreters, but refer themselves as compilers, probably due to their linking functionalities.



MERRIMACK COLLEGE

Friendly advice, using an online server as such, always
keep a local copy of your code.

Installing C++ and Java in your machine

- Choose an IDE
 - There are several options, I personally like Visual Studio Code that is free, available to all Operating Systems, and reasonably light compared to other options
 - VSCode official page: <https://code.visualstudio.com>
- Install the proper compiler according to your Operating System (Windows, MacOS, or Linux)
 - Video exemplifying the setup C++ in a Windows system [here](#)
 - Video exemplifying the setup C++ in a MacOS system [here](#)
 - Video exemplifying the setup Java in a Windows system [here](#)
 - Video exemplifying the setup Java in a MacOS system [here](#)



Visible Differences

A Battleship named Java.
A Submarine named C.
An Yacht named Python.

Python, Java and C/C++ are similar, but there are some visible difference between them, some as simple as syntax differences, others as complex as completely different commands and structure.

In general those differences reflect the purpose and history of the languages. They can be seen in terms of:

- Program Structure
- Basic Commands
- Classes, Objects, etc.



History and Purpose

	C	C++	Java	Python
Creation and big revisions	1972 1990, 1999, 2017	1985 1998, 2011	1995 2006, 2017...	1991 2000, 2008
Initial Purpose	Simple code	Add OO to C	Bring OO to the industry	Solve practical problems fast
Strongest Suit	Very close to hardware	Performance and flexibility	Facilitate code reuse	Fast prototyping
Market Share	Embedded Systems	Classical CS, Academia	Big industry, Large projects	Open source, Artificial Intel.
<i>Trend</i>	<i>Stable</i>	<i>Stable</i>	<i>Declining</i>	<i>Growing</i>



MERRIMACK COLLEGE

Wikipedia: [C](#) - [C++](#) - [Java](#) - [Python](#)

Program Structure

- In **Python**
 - Mostly packages, sometimes scripts, sometimes a main function, sometimes classes
- In C
 - Packages, a main function calling everything else
- In C++
 - Packages, a main function calling everything else, and frequently classes
- In Java
 - Packages and Classes

```
# Python program for implementation of Bubble Sort

def bubbleSort(arr):
    n = len(arr)

    # Traverse through all array elements
    for i in range(n):

        # Last i elements are already in place
        for j in range(0, n-i-1):

            # traverse the array from 0 to n-i-1
            # Swap if the element found is greater
            # than the next element
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]

# Driver code to test above
if __name__ == "__main__":
    arr = [5, 1, 4, 2, 8]

    bubbleSort(arr)

    print("Sorted array is:")
    for i in range(len(arr)):
        print("%d" % arr[i], end=" ")
```

Blocks by
indentation



Program Structure

- In Python
 - Mostly packages, sometimes scripts, sometimes a main function, sometimes classes
- In C
 - Packages, a main function calling everything else
- In C++
 - Packages, a main function calling everything else, and frequently classes
- In Java
 - Packages and Classes

```
// C program for implementation of Bubble sort
#include <stdio.h>

void swap(int* xp, int* yp)
{
    int temp = *xp;
    *xp = *yp;
    *yp = temp;
}

// A function to implement bubble sort
void bubbleSort(int arr[], int n)
{
    int i, j;
    for (i = 0; i < n - 1; i++)

        // Last i elements are already in place
        for (j = 0; j < n - i - 1; j++)
            if (arr[j] > arr[j + 1])
                swap(&arr[j], &arr[j + 1]);
}

/* Function to print an array */
void printArray(int arr[], int size)
{
    int i;
    for (i = 0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

// Driver program to test above functions
int main()
{
    int arr[] = { 5, 1, 4, 2, 8 };
    int n = sizeof(arr) / sizeof(arr[0]);
    bubbleSort(arr, n);
    printf("Sorted array: \n");
    printArray(arr, n);
    return 0;
}
```

Blocks by
delimitation

```
;  
{ }
```



Program Structure

- In Python
 - Mostly packages, sometimes scripts, sometimes a main function, sometimes classes
- In C
 - Packages, a main function calling everything else
- In **C++**
 - Packages, a main function calling everything else, and frequently classes
- In Java
 - Packages and Classes

```
// C++ program for implementation
// of Bubble sort
#include <bits/stdc++.h>
using namespace std;

// A function to implement bubble sort
void bubbleSort(int arr[], int n)
{
    int i, j;
    for (i = 0; i < n - 1; i++)

        // Last i elements are already
        // in place
        for (j = 0; j < n - i - 1; j++)
            if (arr[j] > arr[j + 1])
                swap(arr[j], arr[j + 1]);
}

// Function to print an array
void printArray(int arr[], int size)
{
    int i;
    for (i = 0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;
}

// Driver code
int main()
{
    int arr[] = { 5, 1, 4, 2, 8};
    int N = sizeof(arr) / sizeof(arr[0]);
    bubbleSort(arr, N);
    cout << "Sorted array: \n";
    printArray(arr, N);
    return 0;
}

// This code is contributed by rathbhupendra
```

Blocks by
delimitation

```
;  
{ }
```



Program Structure

- In Python
 - Mostly packages, sometimes scripts, sometimes a main function, sometimes classes
- In C
 - Packages, a main function calling everything else
- In C++
 - Packages, a main function calling everything else, and frequently classes
- In **Java**
 - Packages and Classes

```
// Java program for implementation of Bubble Sort
import java.util.*;

class BubbleSort {
    void bubbleSort(int arr[])
    {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++)
            for (int j = 0; j < n - i - 1; j++)
                if (arr[j] > arr[j + 1]) {
                    // swap arr[j+1] and arr[j]
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
    }

    /* Prints the array */
    void printArray(int arr[])
    {
        int n = arr.length;
        for (int i = 0; i < n; ++i)
            System.out.print(arr[i] + " ");
        System.out.println();
    }

    // Driver method to test above
    public static void main(String args[])
    {
        BubbleSort ob = new BubbleSort();
        int arr[] = { 5, 1, 4, 2, 8 };
        ob.bubbleSort(arr);
        System.out.println("Sorted array");
        ob.printArray(arr);
    }
}

/* This code is contributed by Rajat Mishra */
```

Blocks by
delimitation

```
;  
{  
}
```



Basic Commands - decision

- In Python
 - if - elif - else
- In C/C++/Java
 - if - else
 - conditional operator
 - switch - case - default

Exp1 ? Exp2 : Exp3;

```
if(boolean_expression) {  
    // statement(s) will execute if the boolean expression is true  
} else {  
    // statement(s) will execute if the boolean expression is false  
}
```

```
if expression1:  
    statement(s)  
elif expression2:  
    statement(s)  
elif expression3:  
    statement(s)  
else:  
    statement(s)
```

```
switch(expression) {  
    case constant-expression :  
        statement(s);  
        break; //optional  
    case constant-expression :  
        statement(s);  
        break; //optional  
  
    // you can have any number of case statements.  
    default : //Optional  
        statement(s);  
}
```



Basic Commands - variables

- In C
 - Integers and Floating points (different precisions), Pointers, Booleans, Chars, Enumerations, and Arrays
 - Declaration and allocation needed
- In C++/Java
 - Everything in C, plus Strings
 - Declaration and allocation needed
- In Python
 - Integers, Floating points, Booleans, and Lists (including Strings)
 - No declaration or allocation needed

```
long int a = 1048356;
double arr[10];
bool tst = true;
enum = { 'm', 't', 'w', 'h', 'f' }
```



```
int a = 56;
double arr[10];
Bool status = true;
String days = "MonTueWedThuFri";
```



```
fact50 = 30414093201713378043612608166064768844377641568960512000000000000
print(10/3)
3.3333333333333335
```



```
myList = [ 0.5, "half", [1, "divided by", 2]]
```



Basic Commands - loops

- In Python
 - for loop - range - break - continue - pass - else
 - while loop - break - continue - else
- In C/C++
 - for loop - break - continue - goto
 - while loop - break - continue - goto
 - do while loop - break - continue - goto
- In Java
 - for loop - break - continue
 - while loop - break - continue
 - do while loop - break - continue



```
count = 0
while (count < 9):
    print 'The count is:', count
    count = count + 1
```



```
for( int a = 10; a < 20; a = a + 1 ) {
    cout << "value of a: " << a << endl;
}
```



```
int x = 10;

do {
    System.out.print("value of x : " + x );
    x++;
    System.out.print("\n");
}while( x < 20 );
```





Basic Commands - functions

- In Python
 - functions are untyped
 - input parameters can be omitted/default values
 - simple variables passed by value, lists passed by reference (almost always)
 - none, single, or multiple return values/objects
- In C/C++/Java
 - functions are explicitly typed (including `void`)
 - function overload on different parameter sets
 - input parameters passed either by value or reference (programmer choice)
 - none or single return value/object

(since in Java everything is a class, functions are called *methods*)

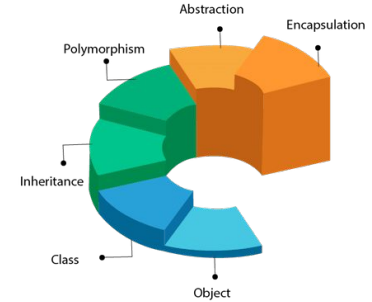
```
def myKids(*kids):  
    print("The youngest kid is", kids[-1])  
    print("The eldest kid is", kids[0])  
    print("I have", len(kids), "kids")  
  
myKids("Karina", "Maria", "Sophia")
```

```
#include <iostream>  
using namespace std;  
  
void myName(string);  
  
int main() {  
    myName("Paulo");  
    return 0;  
}  
  
void myName(string name) {  
    cout << "My name is " << name << endl;  
}
```



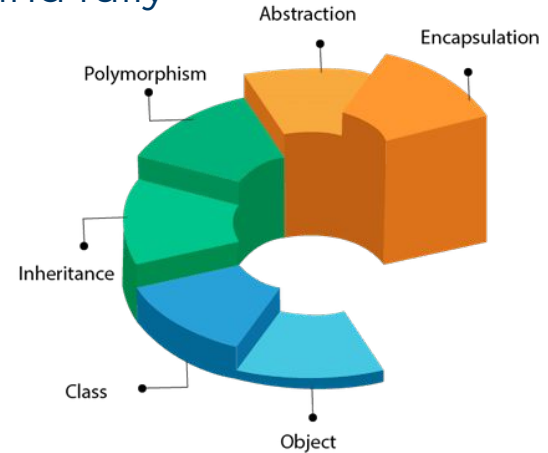
Classes, Objects, etc.

- In **C** there are no Classes, only User Defined Abstract Data Types (Data structures, no methods to handle them).
- The initial goal of **C++** classes was to provide OO concepts without sacrificing performance achieved by C;
 - In 1998 the Standard Template Library - STL was incorporated into C++ enhancing the Classes' support;
 - The definition of Abstract Classes evolved based on Java Interfaces.
- The initial goal of **Python** classes is to provide OO concepts with minimal coding complexity;
 - Python evolution incorporates new features, including OO features.
- **Java** language was defined to follow strictly all aspects of OO concepts;
 - The initial Java interpreters often sacrificed performance to preserve OO concepts, and thus to facilitate software reuse.



Classes, Objects, etc.

- **Objects, Classes**, and **Inheritance** are provided to Python, C++, and Java very similarly (syntax differences);
- **Polymorphism** is partially provided by Python, and fully provided by C++ and Java;
- **Abstraction** can be emulated by Python using methods address as variable data;
- **Abstraction** is available by C++ using Abstract Classes;
- **Abstraction** is available by Java using Interfaces;
- Different levels of **encapsulation** are provided by Python, C++, and Java (the user needs to control it explicitly or refrain from overriding it).



Subtle Differences

The main problems to change programming languages are not the languages.

Most programmers are concern with the syntax differences among programming languages, but the syntax differences are often easily overcome with the help of a supporting IDE. Frequently, the real problems are less visible are rely on:

- Packages available for each language/version
- Documentation and Resources including developer forums
- Internal Implementations issues affecting performance and compatibility

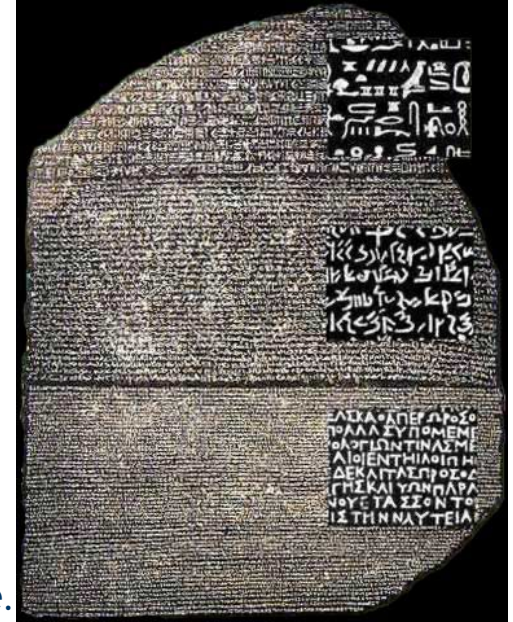


References and Rosetta Stone Examples

When moving to a different language, at least well established ones as Python, C, C++, or Java the language syntax learning is usually a matter of few days and reference sources make the adaptation very easy.

Several websites present classical algorithms implementations in several languages, like the GeeksForGeeks Bubble Sort example in the first slides. Rosetta stone resources like that are often a great way to start the learning of a new language.

The real problems changing from one programming language to another reside on usually what you don't see.



MERRIMACK COLLEGE

Reading: [Matrix Multiplication in C/C++/Java/Python/etc.](#)

Packages Availability

One of the major differences among programming languages rely on the support available by packages. While some standard packages are similar enough in purpose, rarely they provide the exact same service with the same input and output parameters.

In fact, the adaptation of packages is the main obstacle to perform automatic conversion between languages.

For example, Linear Algebra packages are available to almost all languages, but Python, C++, and Java use different ones:

- **numpy** is largely the more popular one for **Python**;
- **Eigen** is a popular option for **C++**, but some applications use **LAPACK++**;
- **Jblas** is a comprehensive alternative for **Java**.



Packages Availability and Versions

The open source availability is changing a lot of the CS industry, especially in emerging Artificial Intelligence areas, but also in more traditional CS applications. Many of Python growth is based on that, as Python seems to be the language of choice for open source developers.

One of the issues with open source development is that the use of specific packages is rarely uniform. This requires keeping track of the employed packages and versions, which is automatically done in Python with the installment of requirements.

Java and C/C++ are less vulnerable to such variety of packages and versions, both by using precompiled packages in C++, and by using pre interpreted package modules in Java.



Documentation and Resources

An important part of developing is to find out support, as frequently details about the language itself, and especially packages needs to be consulted regularly.

Common sources are the official language/packages documentation, but also developer's forums, as GeeksForGeeks, and public open schools, as W3Schools and TutorialsPoint, to site a few.

The online resources are a great help, usually better than asking a friend, but of course, seeking information in the Internet must always be accompanied by caution, as deprecated, and sometimes plainly wrong information is frequently found.



MERRIMACK COLLEGE

A good programmer always seeks for help.

Internal Implementation

With C/C++ the implementation is mostly explicit, except for code optimization level while compiling code. Memory allocation and release is always explicit. The code performance is maximized based on user choices.

With Java all memory allocation is explicit, and memory release is optional. However, the recovery of unused memory (unassigned pointers) is automatically performed: Garbage Collection. Some performance issues are handled by the user, others by the Java machine (interpreter) implementation.

With Python memory allocation is often implicit, and memory release is optional. The performance relies primarily on the Python interpreter implementation, and secondarily on choice of packages.



The algorithm decisions (complexity) always plays an important role for all languages.

This Week's tasks

- Post discussion
- Programming project #2
- Quiz #1

Tasks

- Comment on **which language** you believe will be more important in your future professional life, and **why**.
- **Convert** a simple **C++** program into **Java**.
- Take (available Saturday) **Quiz #1** about the contents of **Week 1** and **Week 2**.



Post Discussion - Week 2 - E#2

Comment on which language you believe will be more important in your future professional life, and why.

- You probably know Python better now, but considering what you saw, or your previous experience, do you believe another language may be more important to you?
 - Feel free to give personal reasons, but relate those with technical characteristics of the language.

Your task:

- Post your discussion in the message board until this Saturday;
- Reply posts of your colleagues in the message board until next Wednesday.



MERRIMACK COLLEGE

This task counts towards the In-Class Exercises grade.

Progr. Project #2 - Week 2 - P#2

Convert this C++ program into a Java program.

A program conversion must reproduce the exact behavior, i.e., the same input in the original and convert programs must deliver the same output.

Run both the original and convert program before submitting!

```
1  #include <iostream>
2  using namespace std;
3
4  bool isPrime(int n) {
5      if (n <= 1)          return false;
6      if (n <= 3)          return true;
7      if (n % 2 == 0 || n % 3 == 0) return false;
8      for (int i = 5; i * i <= n; i += 6) {
9          if (n % i == 0 || n % (i + 2) == 0) return false;
10     }
11     return true;
12 }
13
14 int main() {
15     int number;
16     do {
17         cout << "Enter a positive number (0 or negative to exit): ";
18         cin >> number;
19         if (number <= 0)    break;
20         if (isPrime(number)) cout << number << " is a prime number." << endl;
21         else               cout << number << " is not a prime number." << endl;
22     } while (true);
23     return 0;
24 }
```



Progr. Project #2 - Week 2 - P#2

- You must submit only the Java code (if multiple files, zip it in a single file).
- If you need to add any explanation or messages to me, make it in the code remarks.
- The quality of your code remarks is not relevant for the evaluation, but make sure you write your name in the remarks.
- This C++ code of the program prime.cpp is available [here](#).

Your task:

- Save your program code and submit it in the appropriate Canvas until Next Wednesday.

This assignment counts towards
the Coding Projects grade.



MERRIMACK COLLEGE

Quiz #1 - Available this Saturday - Q#1

- This quiz covers the contents presented in Week 1 and Week 2.
- The quiz is not timed, but the hard deadline to submit it is next Wednesday.
- The quiz is composed by 10 simple questions that you need to answer by choosing the correct answer.
- You can only submit the quiz once, so make sure you double checked your answers before submitting it.
- This quiz is open notes and you can consult any materials referenced in these slides to answer it, but the answer must be your own, not from others.

Your task:

- Submit your quiz in Canvas until Next Wednesday 11:59pm.



MERRIMACK COLLEGE

This assignment counts towards the Quizzes grade.

” Week 2 of CSC 6303

- **Post in the discussions until Saturday, reply until next Wednesday**
 - **Do the coding project until next Wednesday**
 - **Take Quiz #1 until next Wednesday**
-

Next Week - From Python to Go, Swift, and modern languages



MERRIMACK COLLEGE